

4. Solicitudes de red y API de servidor

4.1. Solicitudes AJAX

El funcionamiento «normal» de la web consiste en navegar a través de un conjunto de páginas web a través de **enlaces y formularios**. Mediante los **formularios** se puede **enviar** información al servidor, mientras que los **enlaces** nos permiten **descargar** nuevos contenidos. En ambos casos se produce un **abandono** de la página que se está visitando en ese momento para **sustituirla** por el destino del enlace o la URL del destino del formulario.

Lo anterior implica que cada vez que se desea interactuar con una aplicación web, como, por ejemplo, al añadir un elemento a un carrito de la compra, hay que **recargar** la página que se está visitando. El funcionamiento sería el siguiente:

1. El usuario añade el elemento al carrito mediante un **formulario**.
2. El **navegador envía el formulario** al servidor. Para ello el navegador **abandona la página original**.
3. El **servidor** recibe los datos del formulario y hace el procesamiento correspondiente [actualización de base de datos, por ejemplo]. Cuando termina, **compone una nueva página** con la información del carrito actualizada y la **envía** de vuelta al navegador del cliente.
4. El **navegador** recibe la nueva página, la **carga** y la muestra al usuario.

Esta manera de trabajar tiene una serie de desventajas:

- Se produce una **recarga** de la página: la página desaparece de la vista del navegador y tras un breve tiempo se muestra la página actualizada.
- El **servidor** tiene que **volver a generar la página** completa mediante su sistema de plantillas (página dinámica). Si hay muchas peticiones simultáneas, el tiempo de procesamiento se incrementa y pueden producirse **ralentizaciones o denegaciones de servicio**.
- La **página** se reenvía de manera **completa**, incluyendo datos, HTML, CSS y JavaScript, por lo que el **tamaño** de la descarga es considerable.

Algunos de los inconvenientes comentados pueden minimizarse utilizando algún tipo de **caché**. Otras, como la recarga de la página, no tienen solución. Por eso se desarrolló la tecnología **AJAX**.

AJAX (*asynchronous JavaScript and XML*) es un **conjunto de tecnologías** que se implementaron para permitir que una página web pudiera **enviar y recibir datos del servidor sin necesidad de abandonar la página**. A las peticiones AJAX también se las llama **peticiones en segundo plano**. Inicialmente, la tecnología AJAX utilizaba:

- **XMLHttpRequest**. Es un **objeto JavaScript** que incorpora una serie de propiedades y métodos para **enviar y recibir datos** del servidor desde JavaScript. En la actualidad es posible seguir utilizándolo, aunque ha sido reemplazado casi en su totalidad por **fetch**.
- **XML**. Es el lenguaje que se utilizaba para **intercambiar datos con el servidor**. En la actualidad ha sido sustituido por **JSON**.
- **HTML, CSS, el DOM y JavaScript**. Tecnologías de entorno cliente que se utilizan para **actualizar la página con los nuevos datos** recibidos del servidor.

La mayoría de las páginas modernas incorporan **funciones realizadas con AJAX**, como, por ejemplo:

- Cuadros de búsqueda que muestran sugerencias conforme se escribe.
- Listas de selección que actualizan la lista de ciudades cuando se selecciona una provincia.
- Actualización de tallas de producto disponibles cuando se elige un determinado color.
- Adición o eliminación de productos en un carrito de compra sin necesidad de recargar la página.
- Actualización de gastos de envío al añadir productos al carrito de compra.

En muchas ocasiones, como en la mayoría de aplicaciones web de **comercio electrónico**, se utiliza una **combinación de navegación entre páginas** con actualizaciones a través de **AJAX**. Así, es habitual que las acciones de añadir al carrito o selección de talla y color se realicen a través de AJAX, mientras que cuando se quiere realizar el pedido se navegue a una nueva página.

Si todas las acciones que realiza la aplicación se hacen a través de **AJAX** (con excepción de la carga inicial), se dice que la aplicación es de tipo **SPA**, *single-page application* o **aplicación de una sola página**.

4.2. La API **fetch**

La API **fetch** permite realizar peticiones HTTP a través de **JavaScript**. Se utiliza, por tanto, para realizar peticiones **AJAX**.

La API **fetch** define un conjunto de **objetos** y una **función global** denominada también **fetch**. La función **fetch** permite enviar y recibir datos del servidor de manera más fácil e intuitiva que **XMLHttpRequest**.

! IMPORTANTE

Una de las tareas que puede hacer **XMLHttpRequest** que no puede realizar **fetch** es **mostrar el proceso de carga de un archivo**. Si queremos que nuestra aplicación muestre dicho proceso de carga, deberemos utilizar obligatoriamente **XMLHttpRequest**.

fetch es una **función asíncrona** que **devuelve una promesa**. La razón es que las peticiones a través de la red son **lentas** en comparación con la velocidad de procesamiento de los equipos informáticos. Así, no tiene sentido que el navegador se quede bloqueado esperando que se complete cada petición.

4.3. Peticiones con **fetch**

La función **fetch** admite dos parámetros:

- El **recurso** al que se quiere acceder [obligatorio]. Puede ser:
 - Una **cadena de texto** con la URL del recurso.
 - Un **objeto URL**.
 - Un objeto de tipo **Request**, definido también en la API **fetch**. Si se utiliza un objeto **Request**, no es necesario utilizar el segundo parámetro, ya que dicho objeto permite definir las mismas opciones.
- Un **objeto** con las opciones de la petición [opcional].

Para construir la **URL del recurso** al que se quiere acceder se puede indicar directamente una cadena de texto o utilizar un objeto **URL**. Dicho objeto proporciona una serie de **métodos** para trabajar con URL y acceder a sus **componentes**, como **protocolo**, **puerto** o **parámetros**. Si se desea construir URL con parámetros, es muy útil que incluyan **caracteres especiales**, ya que deben ser **codificados adecuadamente** para su envío. A continuación, puedes ver algunas de las opciones que proporciona el objeto **URL**:

```
var url1 = new URL("https://www.misitio.com/paginas/pagina1.php?id=123&name=name1");

console.log(url1.protocol); // 'https:'
console.log(url1.host); // 'www.misitio.com'
console.log(url1.pathname); // 'paginas/pagina1.php'
console.log(url1.search); // '?id=123&name=name1'
console.log(url1.searchParams.get("id")); // '123'
console.log(url1.searchParams.get("name")); // 'name1'

// Se añade un parámetro con caracteres no ASCII
url1.searchParams.append("nuevo", "canción navideña");

console.log(url1.search); // '?id=123&name=name1&nuevo=can-ci%C3%B3n+navide%C3%B1a'
console.log(url1.searchParams.get("nuevo")); // 'canción navideña'
```




EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Enlaces web*, dispones de un enlace en el que se explican con detalle todas las opciones de los objetos **URL**.

Las **opciones** de la petición se deben indicar en un **objeto** JavaScript con una serie de **propiedades**. A continuación se indican las más utilizadas:

- **method**. Define el **método** HTTP que se utilizará en la petición. Si no se indica, se utiliza **GET**. Las opciones más utilizadas son **"GET"**, **"POST"**, **"PUT"**, **"DELETE"** y **"OPTIONS"**.
- **headers**. Define un **objeto** con las **cabeceras** HTTP que se enviarán junto con la petición. Se puede utilizar un objeto literal cuyas propiedades son las cabeceras que se quieren utilizar, o un objeto **Headers** (también definido en la especificación de la API **fetch**).
- **body**. Almacena el **cuerpo** de la petición, es decir, los **datos** que se quieren enviar. Lo más habitual es utilizar una **cadena de texto** con los datos en formato JSON (generada mediante **JSON.stringify**, por ejemplo). También se pueden indicar **datos binarios** o **datos de formulario** en formato **formData**. No es posible enviar datos con peticiones **GET** o **HEAD**.
- **credentials**. Permite indicar si la petición debe incluir las **cookies** o información de autenticación almacenada para el sitio con el que se quiere conectar. **Por defecto, no se incluyen**.
- **signal**. Permite indicar un objeto **AbortSignal** para **abortar** la petición mediante un **AbortController**. Lo estudiaremos más adelante.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Enlaces web*, dispones de un enlace en el que se explican con detalle todas las opciones del método **fetch()**.

4.4. Datos recibidos por **fetch**

Cuando se **resuelve** la promesa devuelta por **fetch**, se produce como resultado un **objeto** de tipo **Response**, que también forma parte de la especificación de la API. Las propiedades del objeto **Response** son:

Tabla 1. Propiedades del objeto **Response**.

Propiedad	Tipo	Descripción
status	Número	Código de respuesta HTTP.
statusText	Texto	Mensaje de estado de la respuesta.
ok	Booleano	true si el código de la respuesta está entre 200 y 299 (códigos de «éxito» en el protocolo HTTP).
type	Texto	Tipo de respuesta.
headers	Objeto Headers	Objeto que almacena las cabeceras de la respuesta.
body	Objeto ReadableStream	Objeto para interactuar con los datos de la respuesta (normalmente, no será necesario utilizarlo).

El objeto **Response** define también un conjunto de **métodos** que devuelven promesas:

Tabla 2. Métodos del objeto **Response**.

Método	Descripción
text()	Devuelve la respuesta en formato texto.
json()	Devuelve datos nativos JavaScript, como resultado de la conversión del formato JSON (el resultado es el mismo que al ejecutar JSON.parse()).
formData()	Devuelve la respuesta en forma de objeto formData .
blob()	Devuelve la respuesta en formato binario.

Normalmente, para recibir los datos de una petición **fetch** será necesario procesar dos promesas:

1. Una primera promesa, que devuelve el objeto **Response**. Una vez recibido dicho objeto, se lanza la segunda promesa, normalmente mediante **Response.json()**.
2. Una segunda promesa, que devuelve los datos de la respuesta.

En la siguiente sección se mostrarán ejemplos de uso de **fetch** para el envío y la recepción de datos.

4.5. Ejemplos de uso con fetch

A continuación, veremos varios ejemplos de uso de **fetch** para realizar peticiones a un servidor web. Realizaremos las peticiones más habituales, que tienen que ver con las operaciones **CRUD** (creación, lectura, actualización y borrado). Para ello, asumiremos que los datos que devuelve el servidor estarán en formato **JSON**.

Con el fin de afianzar los conocimientos de la unidad, se alternará el uso de promesas y **async/await** en los ejemplos.

A. PETICIONES GET

Normalmente, las peticiones de tipo **GET** se suelen realizar para obtener datos del servidor. El caso más sencillo es el siguiente:

```
async function programaPrincipal() {

    let respuesta = await fetch("https://www.miservidor.com/usuarios");
    // Primera promesa resuelta
    // 'respuesta' es un objeto de tipo 'Response'

    if (respuesta.ok) {
        // Si el código de la petición es un código de éxito,
        // devolvemos una segunda promesa con la transformación de los datos a
        JSON
        let datos = await respuesta.json();
        // Segunda promesa resuelta

        // ...
        // Tratamiento de los datos
        // ...
    } else {
        console.log("Error de red");
    }
}

programaPrincipal();
```

Con promesas quedaría de la siguiente manera:

```
fetch("https://www.miservidor.com/usuarios")
    .then(function(respuesta) {
        // Primera promesa resuelta
        // 'respuesta' es un objeto de tipo 'Response'
        if (respuesta.ok) {
            // Si el código de la petición es un código de éxito,
            // devolvemos una segunda promesa con la transformación de los
```

```
datos a JSON
    return respuesta.json();
} else {
    // En caso contrario, se genera un error
    throw("Ha habido un error");
}
}).then(function(datos) {
// Procesamiento de la segunda promesa

// ...
// Tratamiento de los datos
// ...
}).catch(function(error) {
console.log(`Error: ${error.message}`);
});
```

En el caso anterior solo se indica la URL. Como no se indican opciones, el método por defecto es **GET**.

En ocasiones, es necesario realizar peticiones **GET** enviando una **cadena de búsqueda**, como, por ejemplo, **https://www.miservidor.com/usuarios?nombre=Laura**. Por supuesto, es posible indicar la URL completa en el primer parámetro de **fetch**, pero también es posible utilizar un objeto **URL**:

```
let url1 = new URL("https://www.miservidor.com/usuarios");
url1.searchParams.append("nombre", "Laura");

fetch(url1)
// .then(...)
// El resto del código sería igual
```

B. PETICIONES DELETE

Las peticiones **DELETE** suelen ser también muy sencillas: se suelen enviar **sin datos en el cuerpo**, y el servidor suele responder también sin datos, indicando únicamente el código con el resultado de la petición:

```
fetch(
    "https://www.miservidor.com/usuarios/usuario2",
    {
method: "DELETE"
    })
    .then(function(respuesta) {
if (respuesta.ok) {
    console.log("Borrado con éxito");
    // Si el servidor devuelve datos al realizar una petición 'DELETE'
podríamos
    // procesarlos como en el caso anterior mediante 'respuesta.
json()'
    // Sin embargo, es habitual que simplemente devuelva un código de
éxito
    // sin datos adicionales.
} else {
    // En caso contrario, se genera un error
    throw("Ha habido un error");
}
}).catch(function(error) {
console.log(`Error: ${error.message}`);
});
```

En este caso, sí que se indica un objeto con opciones con la propiedad `method` para indicar que estamos realizando una petición de tipo `DELETE`.

C. PETICIONES PUT

Las peticiones `PUT` suelen utilizarse para actualizar los datos de un recurso, por lo que dichas peticiones suelen incorporar datos en el cuerpo de la petición. El servidor puede responder con una representación de los datos actualizados del recurso, aunque también es posible que devuelva únicamente el código de la respuesta [éxito o error] sin datos adicionales.

```
async function programaPrincipal() {

    // Objeto que almacena los datos del usuario que se va a actualizar
    let usuario1 = {
nombre: "Laura",
apellidos: "Pérez"
    };

    let respuesta = await fetch(
"https://www.miservidor.com/usuarios/usuario1",
    {
        // Método 'PUT'
        method: "PUT",
        // Cabeceras
        headers: {
            // Los datos se van a enviar en formato JSON
            'Content-Type': 'application/json;charset=utf-8'
        },
        // Datos convertidos a formato JSON
        body: JSON.stringify(usuario1)
    });
    // Primera promesa resuelta
    // 'respuesta' es un objeto de tipo 'Response'

    if (respuesta.ok) {
console.log("Petición PUT realizada con éxito.");
// Si el código de la petición es un código de éxito,
// devolvemos una segunda promesa con la transformación de los datos a
JSON
        let datos = await respuesta.json();
        // Segunda promesa resuelta

        // ...
        // Tratamiento de los datos
        // ...
    } else {
console.log("Error de red");
    }
}

programaPrincipal();
```


D. PETICIONES POST

Las peticiones **POST** suelen utilizarse para crear nuevos recursos, por lo que dichas peticiones **suelen incorporar datos en el cuerpo de la petición**. El servidor puede responder con una representación del recurso creado, aunque también es posible que devuelva únicamente el código de la respuesta (éxito o error) sin datos adicionales.

```

async function programaPrincipal() {

    // Objeto que almacena los datos del usuario que se va a añadir
    let usuario1 = {
nombre: "Inés",
apellidos: "Martínez"
    };

    let respuesta = await fetch(
"https://www.miservidor.com/usuarios",
    {
        // Método 'POST'
        method: "POST",
        // Cabeceras
        headers: {
            // Los datos se van a enviar en formato JSON
            'Content-Type': 'application/json;charset=utf-8'
        },
        // Datos convertidos a formato JSON
        body: JSON.stringify(usuario1)
    });
    // Primera promesa resuelta
    // 'respuesta' es un objeto de tipo 'Response'

    if (respuesta.ok) {
console.log("Petición POST realizada con éxito.");
        // Si el código de la petición es un código de éxito,
        // devolvemos una segunda promesa con la transformación de los datos a
JSON
        let datos = await respuesta.json();
        // Segunda promesa resuelta

        // ...
        // Tratamiento de los datos
        // ...
    } else {
console.log("Error de red");
    }
}

programaPrincipal();

```



4.6. Objeto formData

En los ejemplos anteriores hemos supuesto que los datos de las peticiones se envían en formato JSON. Para ello es de suponer que el código de servidor espera los datos en dicho formato. Para indicar al servidor el formato en que enviamos los datos, utilizamos la cabecera HTTP **Content-Type**. En el caso de JSON, se utiliza **Content-Type=application/json**.

Existen casos en que el código del servidor espera que los datos estén codificados en otro formato. Uno de los más utilizados aparte de JSON es **multipart/form-data**. Este formato de datos se utiliza con **formularios HTML** y además **permite el envío de ficheros** (el formato **application/json** no lo permite).

Para trabajar con este tipo de datos, el navegador ofrece la interfaz **FormData**. A través de ella es posible acceder a los datos de un formulario y enviarlos mediante **fetch** con la codificación **multipart/form-data**.



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Ejemplos para trabajar y analizar*, dispones del archivo **DWEC_U04_A04_01.html**, con el código de ejemplo.

Como puede verse en el ejemplo anterior, al indicar dentro de la opción **body** un objeto de tipo **FormData** no es necesario indicar la cabecera **Content-Type**, ya que se da por supuesta.

4.7. Abortar peticiones

Es posible abortar una petición **fetch** una vez que se haya lanzado. Para ello se debe crear un objeto **AbortController** y pasar su propiedad **signal** en la opción **signal** de **fetch**. Si se desea abortar la petición, se deberá ejecutar el método **abort()** del controlador **AbortController**. Si la petición se aborta, la promesa que devuelve **fetch** se rechaza. A continuación se muestra un ejemplo:

```
// Controlador para abortar
const controlador = new AbortController()

fetch("http://misitio.com/usuarios",
  {
    // Se pasa la propiedad 'signal' del controlador
    // a la opción 'signal' de 'fetch'
    signal: controlador.signal
  })
  .then(response => {
    // Tratar respuesta
  })
  .catch(error => {
    // Si la petición se aborta, se genera un error 'AbortError'
    if (error.name == "AbortError") {
      console.log("Petición cancelada");
    }
  });

// Abortar la petición
controlador.abort();

// Petición cancelada
```


! IMPORTANTE

AbortController puede utilizarse con otras funciones asíncronas, no solo con **fetch**. Para ello, dichas funciones deben implementar una manera de recibir la señal del **AbortController**.

4.8. Trabajo con API de servidor

En muchas de las aplicaciones actuales la lógica de cliente y la de servidor están **desacopladas**: por un lado está el **cliente**, que suele ser una aplicación **SPA** y suele alojarse en un servidor de archivos estáticos o un **CDN**; por otro lado está la **lógica de servidor**, que suele estar diseñada en forma de **API** con unos **puntos de destino** (URL) con los que se interactúa mediante los **métodos** del protocolo HTTP (**GET**, **POST**, **PUT** y **DELETE**). A este tipo de arquitectura se le suele denominar **arquitectura REST**.

! IMPORTANTE

La arquitectura **REST** es mucho más que utilizar métodos HTTP sobre una API web.

En muchas ocasiones, las API de las aplicaciones están **abiertas** o por lo menos **permiten el acceso** con una serie de restricciones (como, por ejemplo, un número máximo de peticiones a la hora). Así, es posible diseñar clientes web que utilicen esas API para utilizar sus funciones. Esto es muy habitual en servicios como Facebook, Twitter, Instagram o Google: podemos crear una web que muestre los últimos *tweets* de un usuario o muestre la información de localización de un negocio.

Sin embargo, existe una directiva de seguridad en los navegadores que **impide a una página web** cargar contenido de un **dominio web distinto** (que no sea mediante etiquetas ****, **<video>** o **<script>**). Así, si se realiza una petición **fetch** para descargar contenido de un recurso alojado en un dominio web distinto se **generará un error**.

Para **permitir el acceso** entre peticiones de diferente dominio se desarrolló el mecanismo **CORS**, *cross-origin resource sharing* o intercambio de recursos de origen cruzado. Este mecanismo permite que los **servidores** puedan servir contenido a sitios web de un **dominio diferente al suyo**. Esta configuración se realiza en la **lógica de servidor** y permite definir una **lista de dominios con acceso** o **permitir el acceso desde cualquier dominio**. A nivel interno, hace uso del método **OPTIONS** y la cabecera **origin** del protocolo HTTP para comprobar el tipo de petición realizada y determinar si el cliente dispone de permisos.

En resumen:

- Las aplicaciones web desacopladas suelen alojar la lógica de cliente y la lógica de servidor en **diferentes servidores**. Dichos servidores pueden ser accesibles a través del **mismo o distinto nombre de dominio**.
- Si la lógica de cliente y la de servidor son accesibles mediante el **mismo nombre de dominio**, las peticiones con **fetch** se pueden realizar sin problemas.
- Si utilizan **distinto nombre de dominio**, es necesario **activar CORS** en la lógica de servidor para permitir el acceso.

4.9. Alternativas a fetch

La API **fetch** está definida únicamente en los **navegadores**, por lo que **no está disponible en NodeJS**. Como **alternativa**, existe una **librería** muy popular, denominada **axios**, que puede utilizarse **tanto en navegador como en NodeJS**.

axios es una librería que también utiliza **promesas**. Su funcionamiento es parecido a **fetch**, aunque define sus propios métodos y **no necesita utilizar dos promesas** para obtener los datos.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Enlaces web*, dispones de un enlace a la documentación de **Axios**.